Day 56

* Constructors in Python :-

→ The purpose of Constructors in Python is that "to initialize the object"

→ Initializing the object is nothing but placing our own data in object without leaving object empty.

Definition of Constructor

→ A constructor is one of the special method which is automatically / implicitly called by PVM during object creation and whose purpose is to initialize the object without leaving the object empty.

syntax:

```
def __init__(self, list of formal params if any):
    Block of statements
    Performs initialization
```

* Rules or properties of Constructors :-

1. The name of the constructor is always def __init__(self,...)
2. Constructors will be called PVM automatically/implicitly during object creation.
3. Constructors will not return any value expect None
4. In python, constructors can be participate in Inheritance process.
5. In python, constructors can be overridden

* Types of Constructors :-

→ In python programming, we have two types of constructors. they are 1) Default or parameter less constructor

2) Parameterized constructor

1) Default or parameter less constructor:-

→ A default or parameter less constructor is one, which never takes any formal parameters except self.

→ The purpose of Default or parameter less constructor is that "to initialize multiple objects of same class with same values".

syntax).

```
def __init__(self):
    Block of statmnts --- Performs initialization process
```

e.g).

```
# Program for demonstrating Default constructor
class Test:
    def __init__(self):
        print("i am from default constructor:")
        self.a=10
        self.b=20
        print("\ta={}\tb={}".format(self.a, self.b))
# main program
t1=Test()
t2=Test()
t3=Test()
# Object creation calls default constructor
```

2) Parameterized constructor:-

→ A parameterized constructor is one, which always takes formal parameters after self.

→ The purpose of parameterized constructor is that "To initialize multiple objects of same class with Different values".

syntax)

```
def __init__(self, list of formal params):
    Block of stmnts --- perform initialization process.
```

e.g) # program for demonstrating parameterized constructor.

```
Class Test:
    def __init__(self, k, v):
        print("i am from parameterized constructor:")
        self.a = k
        self.b = v
        print("\ta={}\tb={}".format(self.a, self.b))

#main program
t1 = Test(10, 20)
t2 = Test(100, 200)
t3 = Test(1000, 2000)
```

object creation calls parameterized constructor

most imp points

Note:- In Class of python, we can't define both Default and parameterized constructors bcoz pvm can remember only latest constructor (due to its interpreatation process).
To full fill the need of both default and parameterized constructors, we define single constructor with default parameter mechanism.

e.g) # program for demonstrating parameterized & Default constructor.

```
Class Test:
    def __init__(self, k=1, v=2): # default & parameterized
        print("i am from default / default parameterized constructor:")
        self.a = k
        self.b = v
        print("\ta:{}\tb={}".format(self.a, self.b))
#main program
t1 = Test()      # object creation calls default constructors.
t2 = Test(10, 20)  # parameterized constructors
```

Non constructor

```
class Employee:
    def getempdata(self):
        self.eno = 10
        self.ename = "Rossum"

#main program
e1 = Employee() # object creation
print("Content of e1 =", e1.__dict__) # {}
e1.getempdata() # we are calling the method explicitly
print("Content of e1 =", e1.__dict__) # {...}
```

Constructor

ex1).

```
class Employee:
    def __init__(self): # Constructor
        print("I am from Constructor")
        self.eno = 10
        self.ename = "Rossum"

#main program
e1 = Employee() # object creation --- pvm calls constructor
print("Content of e1 =", e1.__dict__) # {}
```

ex2).

```
def __init__(self, eno, ename) : # Constructor कहलाता है.
    फिट बाकी सब same.

#main program
e1 = Employee(100, "Rossum") —
    फिट बाकी सब same.
e2 = another object भी बनाएंगे.
```

Default param cnst

```
class Test:
    def __init__(self, a=1, b=2):
        print("Default / parameterised")
        self.a = a
        self.b = b
        print("---")
        print("val of a : {}".format(self.a))
        print("val of b : {}".format(self.b))

#main program
t1 = Test() # object creation pvm calls default constructor
t2 = Test(10, 20) # object creation pvm call parametrized constructor
```

t4 = Test(b=1000)
t5 = Test(b=1000, a=2000)

Q. write a python program which will accept a student details such as student number, name, and marks in 3 subject.
Calculate the total marks, percentage, and decide the grade.
Store all these details in the student table in mySQL.

mySQL > create table result (rno int, name varchar (10), cm int, cpp int, pythm, int, totmarks int, percent float, grade varchar(10))

```
# Validation of student number.
import mysql.connector
class studentMarksReport:
    def __init__(self):
        while(True):
            self.sno=int(input("Enter student Number:"))
            if(self.sno>0):
                break
            print("\t{} is invalid student Number:",format(self.sno))
        # Accept student Name
        self.sname=input("Enter student Name:")
        # Validation of clang marks(0-100)
        while(True):
            self.cm=int(input("Enter Marks in C:"))
            if(self.cm>=0) and (self.cm<=100):
                break
            print("\t{} is invalid marks in c:".format(self.cm))
        # Validation of cpp lang marks (0-100).
        if(0<=self.cppm<=100):
            बाकी सब same.
        # Validation of python lang marks (0-100).
        > PYm. Baki sab same.
    def compute(self):
        # Compute totalmarks and percentage
        self.totmarks=self.cm+self.cppm+self.pym
        self.percent=(self.totmarks/300)*100.
        # Decide the grade
        if(self.cm<40) or (self.cppm<40) or (self.pym<40):
            self.grade="fail".
```

```
else:
    if (self.totmarks >= 250) and (self.totmarks <= 300):
        self.grade = "Distinction
```

>200 first — <=249

>150 second. — <199

>120 third — <149

```
def dispmarksreport(self):
    # Display student marks report
    print("-----")
    print("\t\t student marks report :")
    print("  -  -  ")
    print("\t student number: {}".format(self.sno))
                    name
                marks in c
                    c++
                    python
    print("  -  -  ",
          student totalmarks
          student percentage ofmarks.
          student grade
```

```
def saverecord(self):
    try:
        # write PDBC code
        con = mysql.connector.connect(host = "127.0.0.1",
                                      user = "root",
                                      passwd = "root",
                                      database = "batch6pm")
        cur = con.cursor()
        iq = "insert into result values (%d, '%s', %d, %d, %d, %d, %f, %s)"
        cur.execute(iq
            %(self.sno, self.sname, self.cm, self.cppm, self.pym,
              self.totmarks, round(self.Percent, 2), self.grade))
        con.commit()
        print("{} student record inserted verify".format(cur.rowcount
    except mysql.connector.DatabaseError as db:
        print("Problem in mysql", db)
```

```
#main program
so = studentmarksReport()
so.compute()
so.dispmarksreport()
so.savesstddata()
```

* objects in python :-

→ When we define a class, memory space is not created for data members and methods but whose memory is created when we create an object w.r.t class name.

→ The purpose of creating an object is that "to store Data"

→ To do any Data processing, it is mandatory to create an object

→ To create an object, there must exists a class definition otherwise we get NameError.

Definition of object

. => Instance of a class is called object (Instance is nothing but allocating sufficient memory space for the Data members and methods of a class).

Syntax)

```
Varname = classname()
        (or)
Varname = classname(val1, val2 ... val-n)
```

e.g) Create an object of student

```
so = student()
```

e.g) create an object Employee

```
eo = Employee(10, "Rossum")
```

* Differences Between Classes and objects

Class

1) A class is a collection of Data members and methods
2) When we define a class, memory space is not created for Data members and methods and it can be treated as specification /model for real time Application
3) Definition of a particular class exists only once.
4) When we develop any program with oops principles, class defination loaded first in main memory only once.

objects

1) instance of a class is object.
2) when we create an object, we get the memory space for data members and methods of class.
3) w.r.t one class Definition, we can create multiple objects.
4) we can create an object after loading the class definition otherwise we get NameError.

Day (73)

* Destructors in python & Garbage Collector

→ we know that Garbage Collector is one of the in-built program in python, which is running behind of every python program and whose is role is to collect un-wed memory space and it improves the performance of python based application.

→ Every Garbage collector program is internally calling its own Destructor functions.

→ the destructor function name in python is def__del__(self).

→ By default, the Destructor always called by Garbage Collector when the program execution completed for de-allocating the memory space of objects which are wed in that program. where as constructor called by pvm implicitly during

object is creation for initializing the object.

→ when the program execution is completed, GC calls its own destructor to de-allocate the memory space of objects presents in program and it is called automatic Garbage collection

→ Hence, we have three programming conditions for calling GC and to make the Garbage collector to call destructor function.

a) By default (or) Automatically GC calls destructor, when the program execution completed (called Automatic Garbage collection)

b) make the object reference as None for calling forceFull Garbage collection (called Forcefull Garbage collection)
syn). objname = None.

c) delete the object by using del operator for calling Forcefull Garbage collection (called forcefull Garbage collection
syntax). del objname

→ Syntax)

```
def __del__(self):
    _ _ _ _
    _ _ _ _
```

→ No need to write destructor in class of python bcoz GC contains its own destructor.

* Garbage collector

→ Garbage collector contains a pre-defined module called "gc".
→ Here gc contains the following functions.

1) isenabled()
2) enable()
3) disable()

→ Gc is not under the control of programmer but it always maintained and managed by OS and PVM

Note: Python programmer need not to write destructor method/function and need not to deal with Garbage collection process by using gc module bcoz PVM and OS takes care about Automatic garbage collection process by automatic enabling Gc.

#program for Demonstrating Destructor.

```
non Destructor
class Employee:
    def __init__(self, eno, ename):
        print("I am from parameterized constructor")
        self.eno = eno
        self.ename = ename
        print("Emp Num: {}".format(self.eno))
        print("Emp Name: {}".format(self.ename))
        print("---")
#main program
print("program Execution started")
e1 = Employee(10,"RS")   # } object creation
e2 = Employee(20,"DK")   # } - makes the PVM to call parameterized constructo
print("program Execution Ended:")
```

Destructor

→ is ke baad.

ex1).

```
def __del__(self):
    print("Garbage Collectors calls __del__(self) for DeAllocating mem space")
```

```
#main program bhi same.
time.sleep(10).
```

ex2). Sab same Bas Ye Change. → import करना है। import time,sys

```
def __del__(self):
    print("Garbage Collectors Calls __del__(self) for De-allocating
          memory space")
    global totmemspace
    totmemspace = totemBspace - sys.getsizeof(self)
    print("now available memory space:{}".format(totmemspace
#main program में भी जैसे का वैसा
```

ex3 की बाद).

```
    #to find the memory space of any object - ... sys.getsize
                                                  (objectname
totmemspace = sys.getsizeof(e1) + sys.getsizeof(e2) +
                                  sys.getsizeof(e3)
print("total memory space =", totmemspace)
print("-  -  -")
print("program execution completed")
time.sleep(5)
```

ex3). ex2 का def del पूरा लिखना है।

```
#main program.
e1 = Employee(10,"Rossum")
print("No longer interested in maintaining the memory space
       for e1 object")
time.sleep(5)
e1 = None # GC calls Destructor forcefully - - forceful Garbage
                                              Collection.
e2 = Employee(20,"Travis")
e3 = Employee(30,"Ritchie")
print("-  -  -")
time.sleep(5)
```

ex4). #main program

(same.). e1 की बाद print फिर time.sleep.

```
e1 = None.
e2 = Employee
```

की बाद, फिर print फिर time.sleep फिर e2 = None.

```
execution completed.
time.sleep
```

ex5). None के जगह ex 4) में जो था वो है — e1 = none है उसके जगह del e1 — e2, e3 उसका है

ex6) sab same होगा।.

```
print —
e1 = employee (10, "rossum")
e2 = e1 # Deep Copy
e3 = e1 # Deep Copy
print (e1.__dict__, id(e1))
print (e2.__dict__, id(e2))
print (e3.__dict__, id(e3))
print ("- - - - - - -")
time.sleep (5)
```

ex7). के बाद).

```
print ("No longer intested in maintaining the memory space
time.sleep(5).
e1 = None
print
time.sleep(5)
del e2
del e3
print ("program execution Completed")
time.sleep(5)
```

Program for demonstrating garbage Collector running process

ex1)

```
import gc
print ("program Execution started")
print ("Initially, is Gc running:", gc.isenabled()) # True
a = 100
b = 200
c = a+b
print ("val of a = {}". format(a))
gc.disable()
print ("val of b = {}". format(b))
print ("Now is Gc Running:", gc.isenabled()) # false
print ("sum = {}". format(c))
```

In [1]:
```
#Program for Demonstrating the need of Constructors
#Non-ConstEx1:-
class Employee:
        def getempvals(self): # Instance Method
                self.eno=100
                self.ename="Rossum"
                self.sal=45


#Main Program
eo=Employee() # Created with Empty Data
```

```
print("content of eo=",eo.__dict__) # { }
eo.getempvals() # Explicitly we are calling Instance method
print("content of eo=",eo.__dict__) # {.............. }
```

```
content of eo= {}
content of eo= {'eno': 100, 'ename': 'Rossum', 'sal': 45}
```

In [2]:
```
#Program for Deomstrating Default Constructor
#DefaultConstEx1
class Test:
        def __init__(self): # Default Constructor
                print("-------------------------------------------------")
                print("I am from Default Constructor")
                self.a=10
                self.b=20
                print("Val of a={}".format(self.a))
                print("Val of b={}".format(self.b))
                print("-------------------------------------------------")

#Main program
t1=Test() # Object Creation---Makes the PVM to call Default Constructor
t2=Test() # Object Creation---Makes the PVM to call Default Constructor
t3=Test() # Object Creation---Makes the PVM to call Default Constructor
```

```
-------------------------------------------------
I am from Default Constructor
Val of a=10
Val of b=20
-------------------------------------------------
-------------------------------------------------
I am from Default Constructor
Val of a=10
Val of b=20
-------------------------------------------------
-------------------------------------------------
I am from Default Constructor
Val of a=10
Val of b=20
-------------------------------------------------
```

In [3]:
```
#Program for Deomstrating Default and Parametrized Constructor in single class
#DefaultParamConstEx1
class Test:
        def __init__(self,p=1,q=2): # Acts as Deafult and Parametrized  Construc
                print("-------------------------------------------------")
                print("I am from Default / Parametrized Constructor")
                self.a=p
                self.b=q
                print("Val of a={}".format(self.a))
                print("Val of b={}".format(self.b))
                print("-------------------------------------------------")

#Main program
t1=Test() # Object Creation---Makes the PVM to call Default  Constructor
t2=Test(10,20) # Object Creation---Makes the PVM to call Parametrized Constructo
t3=Test(100) # Object Creation---Makes the PVM to call Parametrized Constructor
t4=Test(q=100) # Object Creation---Makes the PVM to call Parametrized Constructo
t4=Test(q=200,p=300) # Object Creation---Makes the PVM to call Parametrized Cons
```

```
------------------------------------------------
I am from Default / Parametrized Constructor
Val of a=1
Val of b=2
------------------------------------------------
------------------------------------------------
I am from Default / Parametrized Constructor
Val of a=10
Val of b=20
------------------------------------------------
------------------------------------------------
I am from Default / Parametrized Constructor
Val of a=100
Val of b=2
------------------------------------------------
------------------------------------------------
I am from Default / Parametrized Constructor
Val of a=1
Val of b=100
------------------------------------------------
------------------------------------------------
I am from Default / Parametrized Constructor
Val of a=300
Val of b=200
------------------------------------------------
```

In [4]:
```
#Program for Deomstrating Parametrized Constructor
#ParamConstEx1
class Test:
        def __init__(self,p,q): # Parametrized  Constructor
                print("------------------------------------------------")
                print("I am from Parametrized Constructor")
                self.a=p
                self.b=q
                print("Val of a={}".format(self.a))
                print("Val of b={}".format(self.b))
                print("------------------------------------------------")

#Main program
t1=Test(10,20) # Object Creation---Makes the PVM to call Parametrized  Construct
t2=Test(100,200) # Object Creation---Makes the PVM to call Parametrized Construc
t3=Test(1000,2000) # Object Creation---Makes the PVM to call Parametrized Constr
```

```
------------------------------------------------
I am from Parametrized Constructor
Val of a=10
Val of b=20
------------------------------------------------
------------------------------------------------
I am from Parametrized Constructor
Val of a=100
Val of b=200
------------------------------------------------
------------------------------------------------
I am from Parametrized Constructor
Val of a=1000
Val of b=2000
------------------------------------------------
```

In [5]:
```
#Program for Demonstrating the need of Constructors
#ConstEx1
class Employee:
        def  __init__(self):  # Default OR Parameter-Less Constructor
                print("I am from Default Constructor")
                self.eno=100
                self.ename="Rossum"
                self.sal=45
                print("---------------------------------")
                print("Emp Number:{}".format(self.eno))
                print("Emp Name:{}".format(self.ename))
                print("Emp Sal:{}".format(self.sal))
                print("---------------------------------")


#Main Program
eo1=Employee() # Whenever we create an object, I want place our own data without
eo2=Employee() # Whenever we create an object, I want place our own data without
```

```
I am from Default Constructor
---------------------------------
Emp Number:100
Emp Name:Rossum
Emp Sal:45
---------------------------------
I am from Default Constructor
---------------------------------
Emp Number:100
Emp Name:Rossum
Emp Sal:45
---------------------------------
```

In [6]:
```
#Program for Demonstrating the need of Constructors
#ConstEx2
class Employee:
        def  __init__(self,eno,ename,sal): # Parameterized Constructor
                print("I am from Parameterized Constructor ")
                self.eno=eno
                self.ename=ename
                self.sal=sal
                print("---------------------------------")
                print("Emp Number:{}".format(self.eno))
                print("Emp Name:{}".format(self.ename))
                print("Emp Sal:{}".format(self.sal))
                print("---------------------------------")

#Main Program
eo1=Employee(100,"Rossum",45.67) # Object Creation---PVM Calls Corresponding Con
eo2=Employee(200,"Travis",15.45) # Object Creation---PVM Calls Corresponding Con
```

```
I am from Parameterized Constructor
--------------------------------
Emp Number:100
Emp Name:Rossum
Emp Sal:45.67
--------------------------------
I am from Parameterized Constructor
--------------------------------
Emp Number:200
Emp Name:Travis
Emp Sal:15.45
--------------------------------
```

In [7]:
```
#Program for Demonstrating Destructor
#Non-DestEx1
class Employee:
        def __init__(self,eno,ename):
                print("I am from Parameterised Constructor")
                self.eno=eno
                self.ename=ename
                print("Emp Num:{}".format(self.eno))
                print("Emp Name:{}".format(self.ename))
                print("------------------------------------------------")


#Main program
print("Program Execution Started:")
e1=Employee(10,"RS") # Object Creation--Makes the PVM To call Parameterized Cons
e2=Employee(20,"DR") # Object Creation--Makes the PVM To call Parameterized Cons
e3=Employee(30,"TR") # Object Creation--Makes the PVM To call Parameterized Cons
print("Program Execution Ended:")
```

```
Program Execution Started:
I am from Parameterised Constructor
Emp Num:10
Emp Name:RS
------------------------------------------------
I am from Parameterised Constructor
Emp Num:20
Emp Name:DR
------------------------------------------------
I am from Parameterised Constructor
Emp Num:30
Emp Name:TR
------------------------------------------------
Program Execution Ended:
```

In [8]:
```
#Program for Demonstrating Destructor
#DestEx1
import time
class Employee:
        def __init__(self,eno,ename):
                print("I am from Parameterised Constructor")
                self.eno=eno
                self.ename=ename
                print("Emp Num:{}".format(self.eno))
                print("Emp Name:{}".format(self.ename))
                print("------------------------------------------------")
        def __del__(self):
                print("__del__() called by GC to de-allocating memory space")
```

```
#Main program
print("Program Execution Started:")
e1=Employee(10,"RS") # Object Creation--Makes the PVM To call Parameterized Cons
e2=Employee(20,"DR") # Object Creation--Makes the PVM To call Parameterized Cons
e3=Employee(30,"TR") # Object Creation--Makes the PVM To call Parameterized Cons
print("Program Execution Ended:")


#Here GC calls  __del__ (Destructor) at the end of Program Execution--This Proce
```

```
Program Execution Started:
I am from Parameterised Constructor
Emp Num:10
Emp Name:RS
------------------------------------------------
I am from Parameterised Constructor
Emp Num:20
Emp Name:DR
------------------------------------------------
I am from Parameterised Constructor
Emp Num:30
Emp Name:TR
------------------------------------------------
Program Execution Ended:
```

In [9]:

```
#Program for Demonstrating Destructor
#DestEx2
import sys,time
class Employee:
        def __init__(self,eno,ename):
                print("I am from Parameterised Constructor")
                self.eno=eno
                self.ename=ename
                print("Emp Num:{}".format(self.eno))
                print("Emp Name:{}".format(self.ename))
                print("------------------------------------------------")
        def __del__(self):
                global totmem
                print("__del__() called by GC to de-allocating memory space--Obj
                totmem=totmem-sys.getsizeof(self)
                print("Now Available Memory Space=",totmem)

#Main program
print("Program Execution Started:")
e1=Employee(10,"RS") # Object Creation--Makes the PVM To call Parameterized Cons
e2=Employee(20,"DR") # Object Creation--Makes the PVM To call Parameterized Cons
e3=Employee(30,"TR") # Object Creation--Makes the PVM To call Parameterized Cons
totmem=sys.getsizeof(e1)+sys.getsizeof(e2)+sys.getsizeof(e3)
print("ID of e1=",id(e1))
print("ID of e2=",id(e2))
print("ID of e3=",id(e3))
print("TOTAL MEMORY SPACE OF ALL OBJECT=",totmem) # 144
print("Program Execution Ended:")
time.sleep(10)


#Here GC calls  __del__ (Destructor) at the end of Program Execution--This Proce
```

```
Program Execution Started:
I am from Parameterised Constructor
Emp Num:10
Emp Name:RS
-----------------------------------------------
__del__() called by GC to de-allocating memory space
I am from Parameterised Constructor
Emp Num:20
Emp Name:DR
-----------------------------------------------
__del__() called by GC to de-allocating memory space
I am from Parameterised Constructor
Emp Num:30
Emp Name:TR
-----------------------------------------------
__del__() called by GC to de-allocating memory space
ID of e1= 2950186351120
ID of e2= 2950186655952
ID of e3= 2950186656272
TOTAL MEMORY SPACE OF ALL OBJECT= 144
Program Execution Ended:
```

In [10]:

```python
#Program for Demonstrating Destructor --Forceful Garbage Collection
#DestEx3
import time
class Employee:
        def __init__(self,eno,ename):
                print("I am from Parameterised Constructor")
                self.eno=eno
                self.ename=ename
                print("Emp Num:{}".format(self.eno))
                print("Emp Name:{}".format(self.ename))
                print("-----------------------------------------------")
        def __del__(self):
                print("__del__() called by GC to de-allocating memory space")

#Main program
print("Program Execution Started:")
e1=Employee(10,"RS") # Object Creation--Makes the PVM To call Parameterized Cons
print("No Longer Interested in Maintaining Object e1 Memory Space")
time.sleep(4)
e1=None # Making the GC to Destructor Forcefully
time.sleep(3)
e2=Employee(20,"DR") # Object Creation--Makes the PVM To call Parameterized Cons
print("No Longer Interested in Maintaining Object e2 Memory Space")
time.sleep(4)
e2=None
time.sleep(3)
e3=Employee(30,"TR") # Object Creation--Makes the PVM To call Parameterized Cons
print("Program Execution Ended:")
time.sleep(5)
# Here  e3 is collgected by GC at the End of the Program --called --Automatics G
```

```
Program Execution Started:
I am from Parameterised Constructor
Emp Num:10
Emp Name:RS
-----------------------------------------------
__del__() called by GC to de-allocating memory space--Object id= 2950186351120
Now Available Memory Space= 96
No Longer Interested in Maintaining Object e1 Memory Space
__del__() called by GC to de-allocating memory space
I am from Parameterised Constructor
Emp Num:20
Emp Name:DR
-----------------------------------------------
__del__() called by GC to de-allocating memory space--Object id= 2950186655952
Now Available Memory Space= 48
No Longer Interested in Maintaining Object e2 Memory Space
__del__() called by GC to de-allocating memory space
I am from Parameterised Constructor
Emp Num:30
Emp Name:TR
-----------------------------------------------
__del__() called by GC to de-allocating memory space--Object id= 2950186656272
Now Available Memory Space= 0
Program Execution Ended:
```

In [11]:

```
#Program for Demonstrating Destructor ------Forceful Garbage Collection
#DestEx4
import time
class Employee:
        def __init__(self,eno,ename):
                print("I am from Parameterised Constructor")
                self.eno=eno
                self.ename=ename
                print("Emp Num:{}".format(self.eno))
                print("Emp Name:{}".format(self.ename))
                print("-----------------------------------------------")
        def __del__(self):
                print("__del__() called by GC to de-allocating memory space")

#Main program
print("Program Execution Started:")
e1=Employee(10,"RS") # Object Creation--Makes the PVM To call Parameterized Cons
print("No Longer Interested in Maintaining Object e1 Memory Space")
time.sleep(4)
del e1  # Making the GC to Destructor Forcefully
time.sleep(3)
e2=Employee(20,"DR") # Object Creation--Makes the PVM To call Parameterized Cons
print("No Longer Interested in Maintaining Object e2 Memory Space")
time.sleep(4)
del e2  # Making the GC to Destructor Forcefully
time.sleep(3)
e3=Employee(30,"TR") # Object Creation--Makes the PVM To call Parameterized Cons
print("No Longer Interested in Maintaining Object e3 Memory Space")
time.sleep(3)
del e3 # Making the GC to Destructor Forcefully
time.sleep(3)
print("Program Execution Ended:")
```

```
Program Execution Started:
I am from Parameterised Constructor
Emp Num:10
Emp Name:RS
-----------------------------------------------
No Longer Interested in Maintaining Object e1 Memory Space
__del__() called by GC to de-allocating memory space
I am from Parameterised Constructor
Emp Num:20
Emp Name:DR
-----------------------------------------------
No Longer Interested in Maintaining Object e2 Memory Space
__del__() called by GC to de-allocating memory space
I am from Parameterised Constructor
Emp Num:30
Emp Name:TR
-----------------------------------------------
__del__() called by GC to de-allocating memory space
No Longer Interested in Maintaining Object e3 Memory Space
__del__() called by GC to de-allocating memory space
Program Execution Ended:
```

In [12]:
```
#Program for Demonstrating Destructor ------Forceful Garbage Collection
#DestEx5
import time
class Employee:
        def __init__(self,eno,ename):
                print("I am from Parameterised Constructor")
                self.eno=eno
                self.ename=ename
                print("Emp Num:{}".format(self.eno))
                print("Emp Name:{}".format(self.ename))
                print("-----------------------------------------------")
        def __del__(self):
                print("__del__() called by GC to de-allocating memory space")

#Main program
print("Program Execution Started:")
e1=Employee(10,"RS") # Object Creation--Makes the PVM To call Parameterized Cons
e2=e1 # Deep Copy
e3=e2 # Deep Copy
print("Program Execution Ended:")
#Here e1,e2,e3 participates in DEEP COPY, So that GC calls Destructor Only Once
```

```
Program Execution Started:
I am from Parameterised Constructor
Emp Num:10
Emp Name:RS
-----------------------------------------------
Program Execution Ended:
```

In [13]:
```
#Program for Demonstrating Destructor ------Forceful Garbage Collection
#DestEx6
import time
class Employee:
        def __init__(self,eno,ename):
                print("I am from Parameterised Constructor")
                self.eno=eno
                self.ename=ename
                print("Emp Num:{}".format(self.eno))
```

```
                print("Emp Name:{}".format(self.ename))
                print("-----------------------------------------------")
        def __del__(self):
                print("__del__() called by GC to de-allocating memory space")

#Main program
print("Program Execution Started:")
e1=Employee(10,"RS") # Object Creation--Makes the PVM To call Parameterized Cons
e2=e1 # Deep Copy
e3=e2 # Deep Copy
print("No Longer Interested in Maintaining Object e1 Memory Space")
time.sleep(4)
del e1  #  GC  won't Destructor Forcefully bcoz still e2 and e3 ponits object
time.sleep(3)
print("No Longer Interested in Maintaining Object e2 Memory Space")
time.sleep(4)
e2=None  # GC  won't Destructor Forcefully bcoz still  e3 ponits object
time.sleep(3)
print("No Longer Interested in Maintaining Object e3 Memory Space")
time.sleep(4)
e3=None  # GC   calls Destructor Forcefully bcoz No Objects Points object
time.sleep(3)
print("Program Execution Ended:")
```

```
Program Execution Started:
I am from Parameterised Constructor
Emp Num:10
Emp Name:RS
-----------------------------------------------
__del__() called by GC to de-allocating memory space
No Longer Interested in Maintaining Object e1 Memory Space
No Longer Interested in Maintaining Object e2 Memory Space
No Longer Interested in Maintaining Object e3 Memory Space
__del__() called by GC to de-allocating memory space
Program Execution Ended:
```

In [14]:

```
#Program for Demonstrating Garbage Collector Existence
#GCEX1
import gc
print("Program Execution Started")
print("Initially, Is GC Running=",gc.isenabled())
a=10
b=20
c=a+b
gc.disable()
print("Is GC Running after disable()=",gc.isenabled())
print("Val of a=",a)
print("Val of b=",b)
gc.enable()
print("Is GC Running after enable()=",gc.isenabled())
print("Sum=",c)
print("Program Execution Ended")
```

```
Program Execution Started
Initially, Is GC Running= True
Is GC Running after disable()= False
Val of a= 10
Val of b= 20
Is GC Running after enable()= True
Sum= 30
Program Execution Ended
```

In [15]:
```
#Program for Demonstrating Destructor
#GCEX2
import sys,time,gc
class Employee:
        def __init__(self,eno,ename):
                print("I am from Parameterised Constructor")
                self.eno=eno
                self.ename=ename
                print("Emp Num:{}".format(self.eno))
                print("Emp Name:{}".format(self.ename))
                print("-----------------------------------------------")
        def __del__(self):
                global totmem
                print("__del__() called by GC to de-allocating memory space--Obj
                totmem=totmem-sys.getsizeof(self)
                print("Now Available Memory Space=",totmem)

#Main program
print("Program Execution Started:")
print("Initially, Is GC Running=",gc.isenabled())
e1=Employee(10,"RS") # Object Creation--Makes the PVM To call Parameterized Cons
e2=Employee(20,"DR") # Object Creation--Makes the PVM To call Parameterized Cons
e3=Employee(30,"TR") # Object Creation--Makes the PVM To call Parameterized Cons
totmem=sys.getsizeof(e1)+sys.getsizeof(e2)+sys.getsizeof(e3)
gc.disable()
print("ID of e1=",id(e1))
print("ID of e2=",id(e2))
print("ID of e3=",id(e3))
print("TOTAL MEMORY SPACE OF ALL OBJECT=",totmem) # 144
print("Is GC Running after disable()=",gc.isenabled())
print("Program Execution Ended:")
time.sleep(10)
```

```
Program Execution Started:
Initially, Is GC Running= True
I am from Parameterised Constructor
Emp Num:10
Emp Name:RS
-----------------------------------------------
I am from Parameterised Constructor
Emp Num:20
Emp Name:DR
-----------------------------------------------
I am from Parameterised Constructor
Emp Num:30
Emp Name:TR
-----------------------------------------------
ID of e1= 2950186350784
ID of e2= 2950186656272
ID of e3= 2950186655952
TOTAL MEMORY SPACE OF ALL OBJECT= 144
Is GC Running after disable()= False
Program Execution Ended:
```

In [ ]: